

Robot Karol

Informatik · Klasse 8 · Praesentationen

Inhaltsverzeichnis

Präsentation – Robot Karol Einstieg	2
Folie 1 – Leitfrage	2
Folie 2 – Die Karol-Welt	2
Folie 3 – Grundbefehle	2
Folie 4 – Vom Algorithmus zum Struktogramm	2
Folie 5 – Problem: zehn gleiche Schritte	2
Folie 6 – Lösung: Wiederholung	2
Folie 7 – Quadrat mit Wiederholung	2
Folie 8 – Fehler sind Informationen	3
Folie 9 – Konkrete Testfälle	3
Folie 10 – Ausblick	3
Quellen	3
 Präsentation – Robot Karol: Bedingungen und Unterprogramme	 4
Folie 1 – Leitfrage	4
Folie 2 – Sensorfragen als Bedingungen	4
Folie 3 – WENN – DANN – SONST	4
Folie 4 – Feste vs. bedingte Wiederholung	4
Folie 5 – Verschachtelung	4
Folie 6 – Wie machen wir Programme übersichtlich?	5
Folie 7 – Teilprobleme bilden – konkret	5
Folie 8 – Eigene Anweisungen definieren	5
Folie 9 – Definition $\neq$ Aufruf	5
Folie 10 – Wiederverwendbarkeit	5
Folie 11 – Bekannten Debugging-Zyklus anwenden	5
Folie 12 – Sicherung	6
Quellen	6
 Präsentation – Karol-Projekt: Muster programmatisch erzeugen	 7
Folie 1 – Projektauftrag	7
Folie 2 – Was soll dein Programm zeigen?	7
Folie 3 – Zerlege das Problem	7
Folie 4 – Prüfe deinen Code	7
Folie 5 – Konkrete Testfragen	7
Folie 6 – So soll das Ergebnis aussehen	7
Folie 7 – Eine mögliche Lösung als Struktogramm	8
Folie 8 – Reflexion am eigenen Code	8
Folie 9 – Transfer	8
Quellen	8

Präsentation - Robot Karol Einstieg

Folie 1 - Leitfrage

Wie wird aus einem Algorithmus ein ausführbares Programm?

Layout-Hinweis: Leitfragenkasten nicht durch Grafiken überdecken. Karol-Welt darunter platzieren.

Folie 2 - Die Karol-Welt

- Karol bewegt sich auf Kacheln.
- Wände, Ziegel und Marken können Teil der Welt sein.
- Karol besitzt Position und Blickrichtung.
- Blickrichtung kann mit Norden, Osten, Süden und Westen beschrieben werden; sie hängt von der jeweiligen Welt ab.

Folie 3 - Grundbefehle

Beispiele: Schritt, LinksDrehen, Hinlegen, Aufheben, MarkeSetzen.

Folie 4 - Vom Algorithmus zum Struktogramm

- Struktogramm = grafische Darstellung eines Algorithmus
- zeigt Sequenz, Wiederholung und später Bedingungen
- Ablauf wird anschließend in Karol-Code übertragen

Folie 5 - Problem: zehn gleiche Schritte

Zehnmal Schritt ist lang und fehleranfällig. Wiederkehrendes Muster erkennen lassen.

Folie 6 - Lösung: Wiederholung

Konkretes Beispiel:

```
wiederhole 10 mal
  Schritt
*wiederhole
```

Folie 7 - Quadrat mit Wiederholung

Konkreter Karol-Code:

```
wiederhole 4 mal
  wiederhole 4 mal
    Schritt
  *wiederhole
  LinksDrehen
*wiederhole
```

Daneben ein passendes Struktogramm zeigen.

Folie 8 - Fehler sind Informationen

Als geschlossenen Lifecycle darstellen: **Beobachten** → **Eingrenzen** → **Ändern** → **Testen** → **Beobachten** → ...

Merksatz: nur eine gezielte Änderung durchführen und danach erneut testen.

Folie 9 - Konkrete Testfälle

- 3 freie Kacheln bis zur Wand → bleibt Karol rechtzeitig stehen?
- Wand direkt vor Karol → kein ungültiger Schritt?
- 6 freie Kacheln → funktioniert derselbe Algorithmus ohne Änderung?

Folie 10 - Ausblick

Karol soll als Nächstes auf seine Umgebung reagieren: Sensorfragen, Bedingungen und eigene Anweisungen.

Quellen

- Robot Karol: <https://www.schule.bayern.de/karol>
- Sächsischer Lehrplan Oberschule Informatik, Klassenstufe 8: <https://www.schulportal.sachsen.de/lplandb/l>
- Grafiken und Struktogramme: eigene didaktische Darstellung.

Präsentation - Robot Karol: Bedingungen und Unterprogramme

Folie 1 - Leitfrage

Wie kann Karol auf unterschiedliche Situationen reagieren und Programmteile wiederverwenden?

Layout-Hinweis: Leitfragenkasten frei halten; Grafik darunter.

Folie 2 - Sensorfragen als Bedingungen

- Sensorfragen liefern Informationen über Karols Umgebung.
- Die hier verwendeten Sensorfragen liefern **WAHR oder FALSCH (JA oder NEIN)**.
- Dadurch können sie direkt als Bedingungen verwendet werden.

Lehrerhinweis: if wertet Bedingungen wahr/falsch aus. Mehrfachauswahlen wie switch/case sind andere Auswahlstrukturen und werden hier nicht benötigt.

Folie 3 - WENN - DANN - SONST

Beispiel:

```
wenn IstWand dann
    LinksDrehen
sonst
    Schritt
*wenn
```

Daneben ein passendes Struktogramm.

Folie 4 - Feste vs. bedingte Wiederholung

Fest: „Gehe genau 5 Felder.“

```
wiederhole 5 mal
    Schritt
*wiederhole
```

Bedingt: „Gehe bis zur Wand.“

```
solange NichtIstWand
    Schritt
*solange
```

Merksatz: **Fest = Wie oft? · Bedingt = Wie lange?**

Folie 5 - Verschachtelung

- Schleife in Schleife
- Bedingung in Schleife
- mehrere Bedingungen und Schleifen können verschachtelt werden
- Einrückung zeigt Zugehörigkeiten

Beispielidee: 4 × 4-Ziegelfeld mit innerer Schleife für eine Reihe und äußerer Schleife für mehrere Reihen.

Folie 6 - Wie machen wir Programme übersichtlich?

Schlecht eingerückten und sauber eingerückten Karol-Code vergleichen.

Beispiel:

```
solange NichtIstWand
  wenn IstZiegel
    Aufheben
  sonst
    Schritt
  *wenn
*solange
```

Folie 7 - Teilprobleme bilden - konkret

Aufgabe: drei Reihen mit je vier Ziegeln bauen. - Teilproblem 1: eine Reihe bauen - Teilproblem 2: zur nächsten Reihe wechseln - Teilproblem 3: Teilprobleme passend wiederholen

Lehrerhinweis: Fachbegriff **Dekomposition** kann ergänzend genannt werden.

Folie 8 - Eigene Anweisungen definieren

Eine Befehlsfolge erhält einen Namen.

```
Anweisung ReiheBauen
  wiederhole 4 mal
    Hinlegen
    Schritt
  *wiederhole
*Anweisung
```

Folie 9 - Definition ≠ Aufruf

- Definition beschreibt den Ablauf.
- Definition wird nicht automatisch ausgeführt.
- Erst ein Aufruf führt den Ablauf aus.

Beispiel:

```
ReiheBauen
LinksDrehen
ReiheBauen
```

Frage: Wie viele Ziegel werden gelegt, wenn ReiheBauen nur definiert, aber nie aufgerufen wird?
→ **keine**.

Folie 10 - Wiederverwendbarkeit

- einmal definieren - mehrfach aufrufen
- weniger kopierter Code
- Änderungen nur an einer Stelle
- Teilaufgaben separat testbar

Folie 11 - Bekannten Debugging-Zyklus anwenden

Nicht erneut vollständig einführen, sondern auf die neue Aufgabe anwenden: **Beobachten** → **Eingrenzen** → **Ändern** → **Testen**

Beispiel: In der zweiten Reihe fehlt ein Ziegel. Welche eigene Anweisung würdest du zuerst prüfen?

Folie 12 - Sicherung

Schüler erklären an einem Codebeispiel: - Bedingung - feste/bedingte Wiederholung - Verschachtelung
- Definition - Aufruf - Wiederverwendung

Quellen

- Robot Karol: <https://www.schule.bayern.de/karol>
- Sächsischer Lehrplan Oberschule Informatik, Klassenstufe 8: <https://www.schulportal.sachsen.de/lplandb/>
- Grafiken, Code und Struktogramme: eigene didaktische Darstellung.

Präsentation - Karol-Projekt: Muster programmatisch erzeugen

Folie 1 - Projektauftrag

Programmiere Karol so, dass er auf einer leeren Fläche ein **4 × 4-Ziegelfeld** erzeugt.

Die Aufgabe steht direkt in der Präsentation; keine externe Datei muss gesucht werden.

Folie 2 - Was soll dein Programm zeigen?

- sinnvolle Wiederholungen
- mindestens eine verschachtelte Wiederholung
- mindestens eine eigene Anweisung für eine wiederkehrende Teilaufgabe
- saubere Einrückung
- verständliche Namen

Folie 3 - Zerlege das Problem

- Wie baust du eine einzelne Reihe?
- Wie wechselst du zur nächsten Reihe?
- Welche Schritte wiederholen sich?
- Welche Teilaufgabe lohnt sich als eigene Anweisung?

Folie 4 - Prüfe deinen Code

Checkliste: - Habe ich eine Wiederholung verwendet? - Habe ich eine verschachtelte Wiederholung verwendet? - Habe ich eine eigene Anweisung definiert? - Wird diese Anweisung mehrfach aufgerufen? - Ist mein Code sinnvoll eingerückt? - Habe ich unnötig kopierten Code vermieden? - Erzeugt mein Programm genau das Zielmuster?

Folie 5 - Konkrete Testfragen

- Werden genau 16 Ziegel gelegt?
- Stimmt die Position nach der ersten Reihe?
- Funktioniert der Reihenwechsel in die richtige Richtung?
- Endet Karol nach der vierten Reihe an der erwarteten Stelle?

Folie 6 - So soll das Ergebnis aussehen

Großes Zielbild des 4 × 4-Ziegelfeldes zeigen.

Lehrerhinweis - kopierfertige Musterlösung

Der endgültige Code muss vor dem Unterricht mit der tatsächlich verwendeten Karol-Version und Startwelt getestet werden. Entscheidend sind Startposition, Blickrichtung und gewünschter Endzustand.

Beispielstruktur:

```
Anweisung ReiheBauen
  wiederhole 4 mal
    Hinlegen
    wenn NichtIstWand dann
      Schritt
    *wenn
```


*wiederhole
*Anweisung

Für den Reihenwechsel eine eigene Anweisung verwenden und passend zur festgelegten Startlage implementieren.

Typische Fehler: - Schritt nach dem letzten Ziegel - falsche Drehrichtung - Reihenwechsel nach der letzten Reihe - eigene Anweisung definiert, aber nicht aufgerufen

Folie 7 - Eine mögliche Lösung als Struktogramm

Schüler sehen die Lösungsstruktur als Struktogramm statt vollständigem Lösungscode: - äußere Wiederholung = Reihen - innere Wiederholung = Felder/Ziegel einer Reihe - Reihenwechsel als eigene Teilaufgabe

Folie 8 - Reflexion am eigenen Code

- Wo ist die äußere Schleife?
- Wo ist die innere Schleife?
- Welche eigene Anweisung hast du definiert?
- Wo wird sie aufgerufen?
- Warum muss die Definition nur einmal geschrieben werden?

Folie 9 - Transfer

- 3 × 6-Feld erzeugen
- Welche Wiederholungszahlen ändern sich?
- Welche eigene Anweisung kann unverändert bleiben?
- Erweiterung: Rahmen- oder Wechselmuster

Quellen

- Robot Karol: <https://www.schule.bayern.de/karol>
- Sächsischer Lehrplan Oberschule Informatik, Klassenstufe 8: <https://www.schulportal.sachsen.de/lplandb/>
- Zielmuster und Struktogramme: eigene didaktische Darstellung.